Dossier d'architecture technique SESAM

Architecture web en n-tiers conteneurisée

Table des matières

1 Objet et périmètre 1
2 Architecture générale et échanges 1
3 Déploiement sur une machine ou une VM 2
4 Interfaces et protocoles utilisés 3
5 Justification des choix techniques 4
6 Versions du socle technique 5
Modules backend 5
7 Environnements et exploitation 5
Configuration 6
Procédure de déploiement proposée 6
Supervision et sauvegardes 6
8 Hébergement sur Internet 6

1 Objet et périmètre

Nous retenons une architecture n-tiers : une interface React, une application backend Spring Boot unique et une base PostgreSQL. Le déploiement cible une machine physique ou une machine virtuelle disposant de Docker. Les conteneurs isolent les composants et rendent leur installation reproductible.

L'interface React permet aux utilisateurs d'accéder aux fonctionnalités de gestion des bâtiments, des salles, des matériels et des comptes. Elle communique avec le backend au moyen d'une API REST utilisant le protocole HTTP et des données au format JSON.

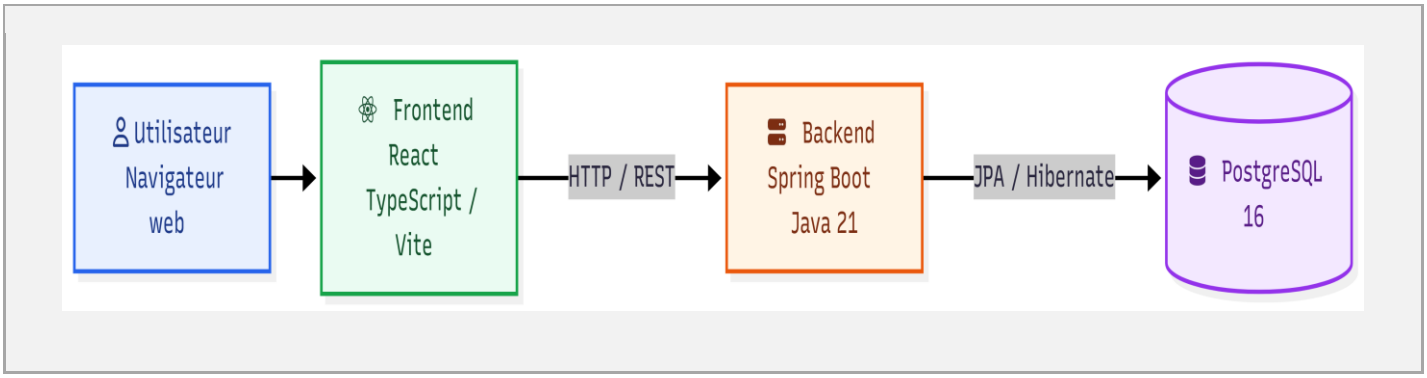
Le backend Spring Boot centralise les règles métier, la gestion des droits d'accès, l'authentification par JWT, la validation des données et les échanges avec la base de données. La base PostgreSQL assure la conservation des informations nécessaires au fonctionnement de l'application.

Ce document décrit les responsabilités de chaque composant, les échanges entre les différentes couches, l'organisation du déploiement Docker et les principaux éléments nécessaires à l'exploitation de l'application. Il sert de référence technique pour comprendre l'architecture retenue et faciliter son installation dans les environnements de développement, de préproduction et de production.

Les schémas présentés dans ce document représentent l'état actuel de l'architecture technique. Ils pourront être ajustés lors des évolutions du projet et de la validation technique finale.

2 Architecture générale et échanges

Le frontend assure la présentation, le backend applique les règles métier et PostgreSQL conserve les données. Les accès à la base passent exclusivement par le backend.



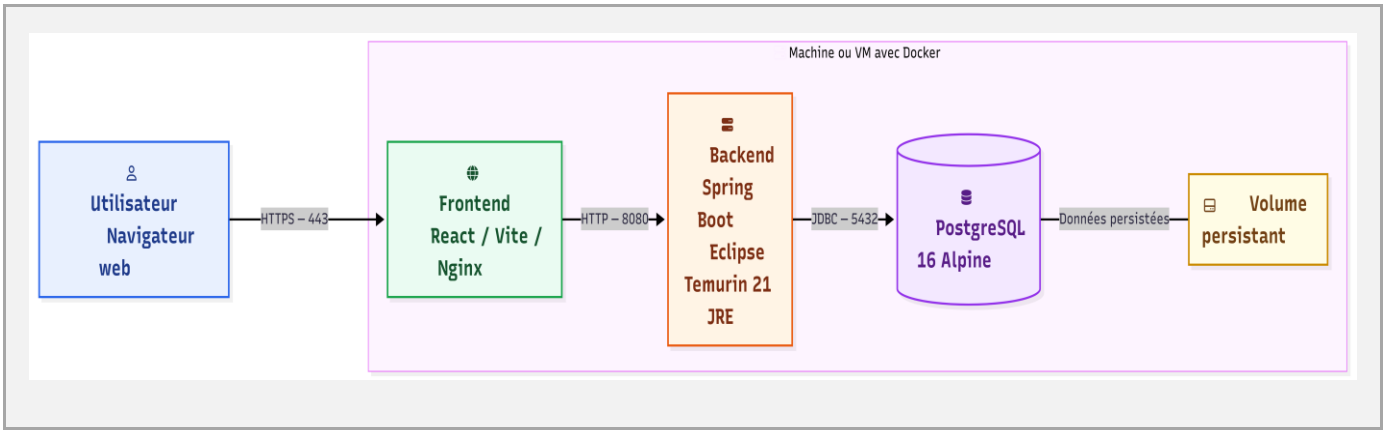
Échange	Contenu et responsabilité
Frontend ↔ backend	Axios réalise les appels à l'API REST. En production, les échanges passent par HTTPS.
Backend ↔ PostgreSQL	Spring Data JPA et Hibernate assurent la persistance. Le pilote PostgreSQL établit la connexion JDBC.

3 Déploiement sur une machine ou une VM

L'application est déployée sur une machine physique ou une machine virtuelle disposant de Docker. Les composants sont séparés dans des conteneurs afin de faciliter leur installation, leur mise à jour et leur exécution dans les différents environnements.

Le conteneur frontend sert les fichiers statiques générés par React et Vite. Il reçoit les requêtes HTTPS des utilisateurs depuis leur navigateur et transmet les appels destinés à l'API vers le backend. Ce conteneur représente le point d'entrée de l'application : les utilisateurs n'accèdent pas directement au backend ou à la base de données.

Le conteneur backend exécute l'application Spring Boot avec le runtime Eclipse Temurin 21 JRE. Il expose l'API REST, applique les règles métier, gère l'authentification et les droits d'accès, valide les données reçues et transmet les opérations de persistance à PostgreSQL. Il ne contient pas les données métier elles-mêmes ; celles-ci sont enregistrées dans la base de données.



Le conteneur PostgreSQL conserve les informations de l'application, telles que les utilisateurs, les bâtiments, les salles, les matériels et les données nécessaires à l'authentification. Un volume Docker persistant est associé à ce conteneur afin que les données restent disponibles après l'arrêt, le redémarrage ou la mise à jour des conteneurs.

Les conteneurs communiquent via un réseau Docker interne. Le frontend transmet les requêtes HTTP au backend sur le port interne 8080. Le backend accède à PostgreSQL au moyen du pilote JDBC sur le port interne 5432. Ces ports ne sont pas destinés à être exposés directement aux utilisateurs.

Seul le frontend est accessible depuis l’extérieur par HTTPS, généralement sur le port 443. Cette organisation limite les accès directs à la base de données et centralise les contrôles de sécurité dans le backend.

La configuration dépend de l’environnement concerné. Les URL de l’API, les identifiants de connexion à PostgreSQL et le secret JWT sont fournis au lancement des conteneurs par des variables d’environnement ou des fichiers de configuration non versionnés. Ces informations ne doivent pas être intégrées au code source ni aux images Docker.

Le déploiement peut être décrit dans un fichier Docker Compose. Ce fichier permet de définir les images utilisées, le réseau interne, le volume PostgreSQL, les variables d’environnement et les dépendances entre services. Il devient alors possible de reproduire une installation similaire en développement, en préproduction et en production, en adaptant uniquement les paramètres propres à chaque environnement.

4 Interfaces et protocoles utilisés

L’application utilise des protocoles distincts selon les composants qui communiquent. Les utilisateurs accèdent à l’interface web via HTTPS. Le frontend communique avec le backend au moyen de requêtes HTTP vers l’API REST. Le backend accède ensuite à la base PostgreSQL à l’aide du pilote JDBC.

Les ports internes Docker ne sont pas destinés à être exposés aux utilisateurs. Ils permettent uniquement les échanges entre les conteneurs du réseau Docker.

Composant ou échange	Version	Port	Protocole utilisé	Rôle
Navigateur → frontend	—	443	HTTPS	Accès sécurisé à l’interface web
Frontend → backend	React 19.2.8	8080 interne	HTTP / REST / JSON	Appels à l’API métier
Backend Spring Boot	4.1.1 / Java 21	8080 interne	HTTP	Exposition de l’API REST
Backend → PostgreSQL	PostgreSQL 16	5432 interne	JDBC / protocole PostgreSQL	Lecture et enregistrement des données
Base de données	postgres:16-alpine	5432 interne	PostgreSQL	Persistance des données

Les valeurs de port indiquées correspondent à l’architecture cible présentée dans le diagramme de déploiement. Seul le port 443 est exposé vers l’extérieur. Les ports 8080 et 5432 restent accessibles uniquement depuis le réseau Docker interne.

5 Justification des choix techniques

Les technologies retenues correspondent aux compétences déjà maîtrisées par notre équipe de développement. Ce choix réduit le temps de prise en main, facilite la maintenance et permet aux développeurs d’intervenir sur l’ensemble des couches de l’application. Il limite également les risques d’erreurs lors du développement de l’application.

L’architecture n-tiers répond à la structure de l’application SESAM. Elle sépare l’interface utilisée par les utilisateurs, les règles métier appliquées par le backend et les données conservées en base. Cette séparation facilite les évolutions futures, car chaque partie peut être modifiée avec un impact limité sur les autres composants.

Choix technique	Justification
React et TypeScript	React permet de construire une interface composée de pages et de composants réutilisables. TypeScript sécurise le développement grâce au typage des données échangées avec l’API. Les équipes connaissent déjà cet environnement.
Vite	Vite fournit un environnement de développement rapide et construit les fichiers statiques destinés au déploiement du frontend.
Spring Boot et Java 21	Spring Boot permet de développer une API REST structurée avec des mécanismes intégrés pour la validation, la sécurité et l’accès aux données. Java est déjà utilisé par l’équipe backend.
Spring Security et JWT	Spring Security centralise la gestion des droits selon les rôles. Les JWT permettent à l’API de vérifier l’identité de l’utilisateur lors de chaque requête protégée.
PostgreSQL	PostgreSQL est une base relationnelle adaptée aux données structurées de SESAM : utilisateurs, bâtiments, salles et matériels. Elle s’intègre directement avec Spring Data JPA et Hibernate.
Docker	Docker permet d’exécuter le frontend, le backend et PostgreSQL dans des environnements séparés mais reproductibles. Il facilite l’installation de l’application sur les différents environnements.
Bootstrap	Bootstrap apporte des composants et des règles de mise en forme réutilisables, ce qui permet de conserver une interface homogène et de réduire le temps consacré au style.

6 Versions du socle technique

Les versions déclarées ci-dessous proviennent des fichiers de configuration du projet. Les valeurs ~ et ^ sont des contraintes npm ; le fichier de verrouillage détermine les versions effectivement installées.

Technologie	Version déclarée	Fonction
React et React DOM	19.2.8	Interface et rendu
TypeScript	~6.0.2	Typage statique
Vite	8.3.0	Développement et build
React Router	7.18.4	Navigation
Axios	^1.20.0	Client HTTP
Bootstrap	^5.3.8	Composants et styles
Bootstrap Icons	^1.13.1	Icônes
ESLint	^10.10.0	Analyse statique
TypeScript ESLint	^8.69.0	Règles TypeScript
Java / runtime	21 / Temurin 21 JRE	Compilation et exécution
Spring Boot	4.1.1	Socle backend
JJWT	0.13.0	Gestion des JWT
PostgreSQL	16 / postgres:16-alpine	Base de données

Modules backend

Spring Web, Spring Data JPA, Spring Validation, Spring Security et Spring Boot Actuator sont les briques retenues. Le pilote PostgreSQL, Lombok, RSQL et Springdoc OpenAPI complètent le backend.

Docker, Git et GitHub complètent le socle. Spring Boot Starter Test est déclaré pour les tests ; les autres outils de test doivent être précisés dans la stratégie de qualité du projet.

7 Environnements et exploitation

Environnement	Branche associée	Usage prévu
Développement	develop	Développement et vérification locale.
Préproduction	preprod	Validation avant mise en production.
Production	main	Version destinée aux utilisateurs.

Les changements sont intégrés de develop vers preprod, puis vers main.

Configuration

Les paramètres propres aux environnements sont externalisés. Les identifiants PostgreSQL et les secrets JWT ne sont pas intégrés au code source ni aux images. Le mécanisme retenu pour injecter ces secrets doit être précisé. Les variables frontend intégrées au build sont publiques et ne doivent pas contenir de secrets.

Procédure de déploiement proposée

1. Identifier une version du code et construire les images frontend et backend avec un identifiant traçable.
2. Préparer la configuration, les secrets, le réseau et le volume de données de l'environnement cible.
3. Sauvegarder la base existante et appliquer les migrations selon une procédure définie.
4. Démarrer les conteneurs et contrôler la disponibilité de la base, de l'API et du frontend.
5. Vérifier la connexion utilisateur et une opération métier avant validation.

Supervision et sauvegardes

Spring Boot Actuator fournit les informations de fonctionnement du backend. Les endpoints exposés doivent être limités, des contrôles de santé Docker doivent être configurés et les journaux applicatifs doivent être conservés selon une durée de rétention définie.

La fréquence et la rétention des sauvegardes, leur emplacement hors de l'hôte, le responsable et le test de restauration doivent être définis. Une procédure de retour à la version précédente doit être prévue et vérifiée avec les migrations de base de données.

8 Hébergement sur Internet

Conformément au point 6.1.6 du Cadre de Cohérence Technique (CCT), relatif à l'hébergement sur Internet, l'hébergement de l'application est prévu sur le réseau C1NP. Ce déploiement implique de disposer d'une pile logicielle à jour et maintenue en condition opérationnelle et de sécurité (MCO/MCS), conformément aux conditions d'homologation déjà applicables aux hébergements actuels.

Le projet est actuellement en cours d'homologation et l'intégration de MindefConnect Internet reste à réaliser. Pour les besoins de démonstration, le scénario d'authentification sera présenté à l'aide d'une page de connexion standard. Ce dispositif provisoire ne se substitue pas à l'authentification MindefConnect Internet exigée pour l'hébergement cible.

Un puits de logs est mis en place afin de centraliser les journaux d'événements applicatifs et de faciliter leur consultation et leur analyse.